

RelayPay Support Agent

Production Customer Support Voice Agent · Week 6 · Emmanuel Aboyeji · 2026-10-03

Live: customer-support-ai-voice-agent.onrender.com · Code: github.com/fullstack-daddy/customer-support-ai-voice-agent

The problem

RelayPay's support team answers the same questions all day — *where is my payout, what are the fees, why is my account under*

review — while the answers sit in a knowledge base and a Postgres database nobody can query by voice. A caller waits on hold for a lookup that takes two seconds.

What it does

A caller speaks to an AI support agent by phone or browser. The agent answers from RelayPay's own knowledge base and database, raises tickets, escalates to humans, and writes every

turn to an audit trail. **It never invents a fact about someone's money.** Anything it cannot ground in a tool result, it declines and routes to a specialist.

How it works

Vapi handles audio, turn-taking and barge-in. It calls **one endpoint** — `/api/vapi` — as an OpenAI-compatible custom LLM. That endpoint runs the **Claude Agent SDK**, which talks to a **custom MCP server** over stdio. Vapi never sees the tools, the database, or the knowledge base.

caller → Vapi (speech) → `/api/vapi` → Claude Agent SDK → MCP server → Supabase

Seven MCP tools: `searchknowledgebase` (pgvector, with a Postgres full-text fallback when no embedding key is set) · `lookupcustomer` · `lookup transaction` · `lookuppayout` · `create supportticket` · `create escalation` · `logconversationevent`.

Safety rails

- **Redaction at the earliest point.** Card numbers (Luhn-checked), CVVs, OTPs and passwords are scrubbed from the caller's turn *before* the model, the tools or any log sees them. 46 tests cover it.
- **The agent cannot send email.** Ticket mail is drafted, then a human edits and approves it in the admin console.
- **The conversation id comes from the server, not the model.** An acceptance run caught the agent inventing one and filing a real ticket against it.
- **Prompt-injection resistant.** No claimed authority, urgency or "this is a test" unlocks a rule; verified by four adversarial probes in `npm run test:agent`.
- **Auth fails closed.** An unset secret denies everyone rather than admitting everyone.

Evidence

9/9 acceptance tests against the live deployment — knowledge grounding, clarifying questions, customer and transaction lookups, ticket creation, escalation, refusals, voice transport and

persistence. One run produced `TICKET-3061627C` and `ESC-E31B9D5B`, 16 turns and 9 tool calls, all correctly linked. **68 unit assertions** plus **34 MCP integration tests** pass. See [Testing evidence](#).

Trade-offs and limits

The Agent SDK spawns a subprocess per call, which is why this runs on Render rather than Vercel — the SDK's native CLI is 238MB against Vercel's 250MB function limit. A warmed session

answers in **5–20s**; the first turn of a cold instance is slower. Retrieval runs on Postgres full-text unless an embedding key is set. Email sending is deliberately gated on a human.

Run it

```
npm install && npm run build
cp .env.example .env      # see docs/deploying.md
npm run dev                # http://localhost:3000
npm test                  # 68 assertions, no API spend
```

Supabase: run the four files in `supabase/migrations/` in order, then `npm run seed`. Vapi setup — including the two secret fields that are configured separately — is in [vapi-configuration.md](#).

Docs

[Testing evidence](#) · [Reflection](#) · [Deploying](#) · [Vapi configuration](#) · [Walkthrough script](#)